

King Saud University
College of Computer and Information Sciences
Department of Information Technology
Big data Project-Final Document



Shopping Behaviors Dataset

Student's Name:	Student's ID:
Shahad basalamah	442200841
Sara alzayed	443203107
Hissah Alhano	443200617

Table of contents

Table of contents	2
1. Introduction	3
2. Dataset Description	3
3. Data Preprocessing	4
3.1 Data Cleaning	4
3.2 Data Integration	4
3.3 Data Reduction	4
3.4 Data Transformation	5
4. RDD Operations	9
4.1 Load CSV to RDD and setup	9
4.2 RDD Transformations and Actions	12
5. SQL Operations	16
6. Machine Learning Operations	20

1. Introduction

This project focuses on analyzing consumer shopping behaviors using a real-world dataset. Understanding customer behavior is crucial for businesses to improve marketing strategies, enhance customer satisfaction, and optimize inventory management. By studying demographic information, purchasing patterns, and feedback, the project aims to uncover insights that can support data-driven decision-making.

The chosen dataset provides a comprehensive view of shopping activities, including customer demographics, product details, transaction data, and customer feedback.

2. Dataset Description

- **Dataset name and source (URL):** Shopping Behaviors Dataset, available on Kaggle – Link <https://www.kaggle.com/datasets/zubairamuti/shopping-behaviours-dataset>
- **Type of data:** Structured data containing numerical and categorical attributes (CSV format).
- **Size:** 3,900 customer records, 18 attributes
- **Key attributes/columns:**
 - Customer demographics: *Customer ID, Age, Gender, Location.*
 - Product details: *Item Purchased, Category, Size, Color, Season.*
 - Transaction details: *Purchase Amount (USD), Payment Method, Shipping Type.*
 - Behavioral data: *Previous Purchases, Frequency of Purchases, Subscription Status, Discount Applied, Promo Code Used.*
 - Customer feedback: *Review Rating.*
- **Reason for choosing this dataset:**
 - It is well-structured and manageable in size, making it suitable for Spark and Scala-based preprocessing.
 - Contains diverse features (demographics, transactions, behavior, feedback) that allow for meaningful analysis.
 - Directly relevant to real-world applications in retail, marketing, and consumer analytics.

3. Data Preprocessing

3.1 Data Cleaning

- Checked for missing values across all 18 columns. The dataset contained no missing values.
- Standardized categorical values, e.g., normalized Yes/No columns (Subscription Status, Discount Applied, Promo Code Used) to ensure consistency (converted to title-case “Yes/No”).
- Ensured numeric fields (Age, Purchase Amount (USD), Review Rating, Previous Purchases) were properly converted to numeric data types to prevent errors in analysis.

```
df.columns = [c.strip() for c in df.columns]

# Normalize Yes/No style columns (title-cased Yes/No)
yn_cols = ["Subscription Status", "Discount Applied", "Promo Code Used"]
for c in yn_cols:
    if c in df.columns:
        df[c] = df[c].astype(str).str.strip().str.title()
        df[c] = df[c].map({"Yes": "Yes", "No": "No"}).fillna(df[c])

# Ensure numeric columns are numeric
numeric_candidates = ["Age", "Purchase Amount (USD)", "Review Rating", "Previous Purchases"]
for c in numeric_candidates:
    if c in df.columns:
        df[c] = pd.to_numeric(df[c], errors="coerce")

print("\nMissing values per column:")
print(df.isna().sum())
```

```
Missing values per column:
Customer ID      0
Age              0
Gender           0
Item Purchased   0
Category         0
Purchase Amount (USD)  0
Location         0
Size             0
Color            0
Season           0
Review Rating    0
Subscription Status  0
Shipping Type    0
Discount Applied  0
Promo Code Used  0
Previous Purchases  0
Payment Method   0
Frequency of Purchases  0
dtype: int64
```

3.2 Data Integration

No external datasets were merged. The dataset is a single source .

3.3 Data Reduction

- Removed the **Customer ID** column, since it is only an identifier and does not provide meaningful information for analysis.
- All other attributes were retained to preserve customer demographics, transaction details, and behavior-related features.

```
df = df.drop('Customer ID', axis=1)
print("Shape after dropping 'Customer ID':", df.shape)
```

```
Shape after dropping 'Customer ID': (3900, 17)
```

3.4 Data Transformation

- Categorical values like **Subscription Status** ("Yes"/"No") and **Gender** ("Male"/"Female") were converted into numerical representations (1 and 0). This step is crucial for ensuring the data is in a format that machine learning algorithms can process effectively.

```
yn_cols = ["Subscription Status", "Discount Applied", "Promo Code Used"]
for c in yn_cols:
    df[c] = df[c].map({"Yes": 1, "No": 0})

df["Gender"] = df["Gender"].map({"Male": 1, "Female": 0})
```

- The columns **Location**, **Category**, and **Payment Method** were transformed using **one-hot encoding**. This technique creates new numerical columns for each unique value, allowing the model to correctly interpret the categorical data. Additionally, the **drop_first=True** parameter was used to prevent multicollinearity, a common issue in machine learning.

```
df = pd.get_dummies(df, columns=["Location", "Category", "Payment Method"], drop_first=True)
```

- Any remaining columns with a boolean data type (True/False) were explicitly converted to their integer equivalents (1/0). This ensures all columns are in a consistent numerical format for analysis and model training.

```
bool_cols = df.select_dtypes(include=['bool']).columns
df[bool_cols] = df[bool_cols].astype(int)
display(df.head())
print(df.columns)
```

	Age	Gender	Item Purchased	Purchase Amount (USD)	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	...	Location_Missouri	Location_Myoming	Category_Clothing	Category_Footwear	Category_Outwear	Payment Method_Cash
0	55	1	Blouse	53	L	Gray	Winter	3.1	1	Express	...	0	0	1	0	0	
1	19	1	Sweater	64	L	Maroon	Winter	3.1	1	Express	...	0	0	1	0	0	
2	50	1	Jeans	73	S	Maroon	Spring	3.1	1	Free Shipping	...	0	0	1	0	0	
3	21	1	Sandals	90	M	Maroon	Spring	3.5	1	Next Day Air	...	0	0	0	1	0	
4	45	1	Blouse	49	M	Turquoise	Spring	2.7	1	Free Shipping	...	0	0	1	0	0	

5 rows x 71 columns

```
Index(['Age', 'Gender', 'Item Purchased', 'Purchase Amount (USD)', 'Size', 'Color', 'Season', 'Review Rating', 'Subscription Status', 'Shipping Type', 'Discount Applied', 'Promo Code Used', 'Previous Purchases', 'Frequency of Purchases', 'Location_Alaska', 'Location_Arizona', 'Location_Arkansas', 'Location_California', 'Location_Colorado', 'Location_Connecticut', 'Location_Delaware', 'Location_Florida', 'Location_Georgia', 'Location_Hawaii', 'Location_Idaho', 'Location_Illinois', 'Location_Indiana',
```

- We converted textual purchase frequency (e.g., Weekly, Fortnightly) into a numeric annual rate to make it model-ready. We created simple intensity ratios—previous purchases per age, spend per previous purchase, and spend per age—to normalize behavior across customers. We encoded season cyclically with sine/cosine to preserve its circular nature, and mapped size to an ordered scale. This produces clearer numeric signals without changing the original categorical encodings done earlier.

```
# 1) Frequency → estimated purchases per year
freq_map = {
    "Daily": 365, "Weekly": 52, "Fortnightly": 26, "Biweekly": 26,
    "Monthly": 12, "Quarterly": 4, "Annually": 1, "Yearly": 1
}
def map_frequency_to_yearly(s):
    return freq_map.get(str(s).strip(), np.nan)

df["Purchases_per_Year"] = df["Frequency of Purchases"].apply(map_frequency_to_yearly)

# 2) Ratios to capture intensity normalized by age/history
df["PrevPurchases_per_Age"] = df["Previous Purchases"] / df["Age"].replace(0, np.nan)
df["Spend_per_PrevPurchase"] = df["Purchase Amount (USD)"] / df["Previous Purchases"].replace(0, np.nan)
df["Spend_per_Age"] = df["Purchase Amount (USD)"] / df["Age"].replace(0, np.nan)

# 3) Season → cyclic (sin/cos)
season_order = ["Spring", "Summer", "Autumn", "Fall", "Winter"]
def season_to_index(s):
    s = str(s).strip()
    if s == "Autumn": s = "Fall"
    return season_order.index(s) if s in season_order else np.nan

season_idx = df["Season"].apply(season_to_index)
df["Season_sin"] = np.sin(2 * np.pi * season_idx / 5)
df["Season_cos"] = np.cos(2 * np.pi * season_idx / 5)

# 4) Size → ordinal
size_map = {"XS": 0, "S": 1, "M": 2, "L": 3, "XL": 4, "XXL": 5}
df["Size_ordinal"] = df["Size"].map(size_map)

# Clean infinities created by divisions
df.replace([np.inf, -np.inf], np.nan, inplace=True)
```

- We kept binary/one-hot columns as is and scaled only continuous features. We used StandardScaler so each scaled feature has mean ≈ 0 and std ≈ 1 , improving comparability and model convergence. Scaled columns were appended with a `_scaled` suffix, while original columns remained for transparency.

```

from sklearn.preprocessing import StandardScaler, MinMaxScaler

# Choose scaler ("standard" or "minmax")
SCALER_CHOICE = "standard"
scaler = StandardScaler() if SCALER_CHOICE == "standard" else MinMaxScaler()

# Continuous features to scale (keep one-hots/binaries as 0/1)
scale_cols = [
    "Age", "Purchase Amount (USD)", "Review Rating", "Previous Purchases",
    "Purchases_per_Year", "PrevPurchases_per_Age", "Spend_per_PrevPurchase",
    "Season_sin", "Season_cos", "Size_ordinal", "Spend_per_Age"
]
scale_cols = [c for c in scale_cols if c in df.columns]

scaled_arr = scaler.fit_transform(df[scale_cols])
scaled_df = pd.DataFrame(
    scaled_arr,
    columns=[f"{c}_scaled" for c in scale_cols],
    index=df.index
)

df_scaled = pd.concat([df.copy(), scaled_df], axis=1)

```

- We verified that all `_scaled` columns were added and inspected the head of the final frame. The sanity check confirms scaling worked: means near zero and unit standard deviations for StandardScaler. Interpreting the sample rows: positive z-scores indicate above-average values for that feature and negative z-scores indicate below-average, while season sine/cosine smoothly reflect seasonal shifts. The dataset is now enriched and numerically well-scaled for distance- and gradient-based models.

```

print("Scaled columns added:", list(scaled_df.columns))
display(df_scaled.head())

# Diagnostics for StandardScaler / MinMaxScaler
if isinstance(scaler, StandardScaler):
    means = scaled_df.mean().round(3).to_dict()
    stds = scaled_df.std(ddof=0).round(3).to_dict()
    print("\nStandardScaler sanity check (means ≈ 0, stds ≈ 1):")
    print("means:", means)
    print("stds :", stds)
else:
    mins = scaled_df.min().round(3).to_dict()
    maxs = scaled_df.max().round(3).to_dict()
    print("\nMinMaxScaler sanity check (range ~ 0..1):")
    print("mins:", mins)
    print("maxs:", maxs)

```

Output:

Scaled columns added: ['Age_scaled', 'Purchase Amount (USD)_scaled', 'Review Rating_scaled', 'Previous Purchases_scaled', 'Purchases_per_Year_scaled', 'PrevPurchases_per_Age_scaled', 'Spend_per_PrevPurchase_scaled', 'Season_sin_scaled', 'Season_cos_scaled', 'Size_ordinal_scaled', 'Spend_per_Age_scaled']

StandardScaler sanity check (means ≈ 0 , stds ≈ 1):

means: {'Age_scaled': -0.0, 'Purchase Amount (USD)_scaled': -0.0, 'Review Rating_scaled': 0.0, 'Previous Purchases_scaled': 0.0, 'Purchases_per_Year_scaled': -0.0, 'PrevPurchases_per_Age_scaled': 0.0, 'Spend_per_PrevPurchase_scaled': -0.0, 'Season_sin_scaled': -0.0, 'Season_cos_scaled': -0.0, 'Size_ordinal_scaled': 0.0, 'Spend_per_Age_scaled': 0.0}

stds : {'Age_scaled': 1.0, 'Purchase Amount (USD)_scaled': 1.0, 'Review Rating_scaled': 1.0, 'Previous Purchases_scaled': 1.0, 'Purchases_per_Year_scaled': 1.0, 'PrevPurchases_per_Age_scaled': 1.0, 'Spend_per_PrevPurchase_scaled': 1.0, 'Season_sin_scaled': 1.0, 'Season_cos_scaled': 1.0, 'Size_ordinal_scaled': 1.0, 'Spend_per_Age_scaled': 1.0}

	Age	Gender	Item Purchased	Purchase Amount (USD)	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	...	Purchase Amount (USD)_scaled	Review Rating_scaled	Previous Purchases_scaled
0	55	1	Blouse	53	L	Gray	Winter	3.1	1	Express	...	-0.285629	-0.907584	-0.785831
1	19	1	Sweater	64	L	Maroon	Winter	3.1	1	Express	...	0.178852	-0.907584	-1.616552
2	50	1	Jeans	73	S	Maroon	Spring	3.1	1	Free Shipping	...	0.558882	-0.907584	-0.162789
3	21	1	Sandals	90	M	Maroon	Spring	3.5	1	Next Day Air	...	1.276716	-0.349027	1.637107
4	45	1	Blouse	49	M	Turquoise	Spring	2.7	1	Free Shipping	...	-0.454531	-1.466141	0.391025

ear_scaled	PrevPurchases_per_Age_scaled	Spend_per_PrevPurchase_scaled	Season_sin_scaled	Season_cos_scaled	Size_ordinal_scaled	Spend_per_Age_scaled
0.397550	-0.841338	-0.164733	-1.120180	0.157375	0.771136	-0.645003
0.397550	-1.152960	2.558377	-1.120180	0.157375	0.771136	1.920609
1.799591	-0.412459	-0.223781	0.211166	1.218176	-1.496910	-0.115443
1.799591	3.498060	-0.352839	0.211166	1.218176	-0.362887	2.899249
-0.950565	0.065339	-0.377556	0.211166	1.218176	-0.362887	-0.511374

4. RDD Operations

4.1 Load CSV to RDD and setup

1. **Path:** We specify the absolute file path to the cleaned CSV we want Spark to read. **Read CSV:** We load the dataset into a Spark DataFrame with headers and inferred data types to avoid manual schema writing.

```
scala> // ===== A)

      val path = "/Users/sara/Downloads/shopping_behavior_cleaned.csv"

      // 2) Read as DataFrame (header + schema inference)
      val dfRaw = spark.read
        .option("header", "true")
        .option("inferSchema", "true")
        .csv(path)

val path: String = /Users/sara/Downloads/shopping_behavior_cleaned.csv
val dfRaw: org.apache.spark.sql.DataFrame = [Age: int, Gender: string ... 87 more fields]
```

2. **Record schema:** We define a case class Record that gives each row a clear, typed structure for downstream RDD work.

```
scala> // 3) Define a typed record (keep human-readable strings; do NOT remap booleans)
[
  case class Record(
    age: Int,
    gender: String,
    item: String,
    category: String,
    amountUSD: Double,
    location: String,
    size: String,
    color: String,
    season: String,
    rating: Double,
    subscription: String,
    shipping: String,
    discount: String,
    promo: String,
    prevPurchases: Int,
    payment: String,
    freq: String
  )
]

class Record
```

3. **Safe parsers:** We set up lightweight helpers (only for the numeric fields we truly need) plus an accumulator to count any rows that fail parsing.

```
scala> // (Optional) drop an "index" column if it exists from previous exports
val df0 = if (dfRaw.columns.contains("index")) dfRaw.drop("index") else dfRaw
val df0: org.apache.spark.sql.DataFrame = [Age: int, Gender: string ... 87 more fields]

scala> // 4) Safe parsers (only parse numerics we truly need)
import scala.util.Try
import org.apache.spark.sql.Row

val badLines = sc.longAccumulator("bad_lines")
val cols = df0.columns.toSet

def getStr(row: Row, name: String): String =
  if (cols.contains(name)) Option(row.getAs[Any](name)).map(_.toString.trim).getOrElse("") else ""

def getIntOpt(row: Row, name: String): Option[Int] =
  if (cols.contains(name)) Try(row.getAs[Any](name).toString.trim.toDouble.toInt).toOption else None

def getDoubleOpt(row: Row, name: String): Option[Double] =
  if (cols.contains(name)) Try(row.getAs[Any](name).toString.trim.toDouble).toOption else None

import scala.util.Try
import org.apache.spark.sql.Row
val badLines: org.apache.spark.util.LongAccumulator = LongAccumulator(id: 184, name: Some(bad_lines), value: 0)
val cols: scala.collection.immutable.Set[String] = HashSet(Location_Illinois, Previous Purchases, Size, Location_Oklahoma, Spend_per_PrevPurchase, Location_Texas, Location_Minnesota, Location_Iowa, Location_New Mexico, Payment_Method_Cash, Spend_per_Age, Location_Wyoming, Payment_Method_Credit Card, Location_Louisiana, Location_Utah, Category_Outerwear, Location_Wisconsin, Purchase Amount (USD)_scaled, Location_California, Location_Missouri, Location_Colorado, Previous Purchases_scaled, Subscription Status, Location_Ohio, Category_Clothing, Item Purchased, Season, Review R...
      badLines.add(1)
```

4. **Row → Record:** We map each DataFrame row to a Record, filling required numeric fields and keeping all other fields as plain strings without any remapping.

```
>.... | badLines.add(1L)

Record(
  age = 0,
  gender = getStr(row, "Gender"),
  item = getStr(row, "Item Purchased"),
  category = getStr(row, "Category"),
  amountUSD = 0.0,
  location = getStr(row, "Location"),
  size = getStr(row, "Size"),
  color = getStr(row, "Color"),
  season = getStr(row, "Season"),
  rating = 0.0,
  subscription = getStr(row, "Subscription Status"),
  shipping = getStr(row, "Shipping Type"),
  discount = getStr(row, "Discount Applied"),
  promo = getStr(row, "Promo Code Used"),
  prevPurchases = 0,
  payment = getStr(row, "Payment Method"),
  freq = getStr(row, "Frequency of Purchases")
)

25/10/04 11:43:47 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
val rddParsed: org.apache.spark.rdd.RDD[Record] = MapPartitionsRDD[15] at map at <console>:2
```

5. **Cache base RDD:** We filter to amountUSD > 0 and cache the resulting base RDD so repeated actions in later parts run faster without recomputation.

```
scala> // 6) Cache because we'll reuse this RDD multiple times (avoids recomputation)
| val base = rddParsed.filter(_.amountUSD > 0.0).cache()
|
val base: org.apache.spark.rdd.RDD[Record] = MapPartitionsRDD[16] at filter at <console>:2
```

6. Output:

```
scala> // 7) Basic sanity actions
val nRows = base.count() // Action #1
val firstRow = base.first() // Action #2
val sample5 = base.take(5) // Action #3

println(s"Rows after basic filter: $nRows, bad lines: ${badLines.value}")
println(s"First row: $firstRow")
println(s"Sample 5: " + sample5.mkString(" | "))

Rows after basic filter: 3900, bad lines: 0
First row: Record(55,Male,Blouse,,53.0,,L,Gray,Winter,3.1,Yes,Express,Yes,Yes,14,,Fortnightly)
Sample 5: Record(55,Male,Blouse,,53.0,,L,Gray,Winter,3.1,Yes,Express,Yes,Yes,14,,Fortnightly) | Record(19,Male,Sweater,,64.0,,L,Maroon,Winter,3.1,Yes,Express,Yes,Yes,2,,Fortnightly) | Record(50,Male,Jeans,,73.0,,S,Maroon,Spring,3.1,Yes,Free Shipping,Yes,Yes,23,,Weekly) | Record(21,Male,Sandals,,90.0,,M,Maroon,Spring,3.5,Yes,Next Day Air,Yes,Yes,49,,Weekly) | Record(45,Male,Blouse,,49.0,,M,Turquoise,Spring,2.7,Yes,Free Shipping,Yes,Yes,31,,Annually)
val nRows: Long = 3900
val firstRow: Record = Record(55,Male,Blouse,,53.0,,L,Gray,Winter,3.1,Yes,Express,Yes,Yes,14,,Fortnightly)
val sample5: Array[Record] = Array(Record(55,Male,Blouse,,53.0,,L,Gray,Winter,3.1,Yes,Express,Yes,Yes,14,,Fortnightly), Record(19,Male,Sweater,,64.0,,L,Maroon,Winter,3.1,Yes,Express,Yes,Yes,2,,Fortnightly), Record(50,Male,Jeans,,73.0,,S,Maroon,Spring,3.1,Yes,Free Shipping,Yes,Yes,23,,Weekly), Record(21,Male,Sandals,,90.0,,M,Maroon,Spring,3.5,Yes,Next Day Air,Yes,Yes,49,,Weekly), Record(45,Male,Blouse,,49.0,,M,Turquoise,Spring,2.7,Yes,Free Shipping,Yes,Yes,31,,Annually))

scala> █
```

- The Scala output shows that the dataset loaded and parsed cleanly: “Rows after basic filter: 3900, bad lines: 0” means all 3,900 rows passed the amountUSD > 0 filter with no parsing failures. The printed First and Sample 5 records confirm the schema is behaving as expected—numeric fields are numeric and categorical fields are preserved as strings (e.g., “Male,” “Yes,” “Express”).

4.2 RDD Transformations and Actions

1) Sizes & Colors

Methodologically, the top sizes and colors were identified using the same core Spark process: a sequence of RDD transformations (**filter**, **map**, **reduceByKey**, **sortBy**) was used to aggregate and rank the data by total spending for sizes and by frequency for colors followed by a **take** action to retrieve the results.

- The analysis shows clear customer preferences. In terms of **sizes**, most spending is concentrated on **medium (M) (105167)** and **large (L) (61667)** sizes, indicating that these are the most commonly purchased items and should be prioritized in inventory planning. Based on this, key suggestions include increasing stock levels for these sizes and showcasing them in promotional materials.

```
def nz(s: String): String = Option(s).map(_.trim).getOrElse("")
val freqMap = Map(
  "Daily" -> 365, "Weekly" -> 52, "Fortnightly" -> 26, "Biweekly" -> 26,
  "Monthly" -> 12, "Quarterly" -> 4, "Annually" -> 1, "Yearly" -> 1
)

val sizeSpend = base
  .filter(r => nz(r.size).nonEmpty)
  .map(r => (nz(r.size), r.amountUSD))
  .reduceByKey(_ + _)
  .map{ case (sz, sum) => (sum, sz) }
  .sortBy(_._1, ascending = false)

val topSizes = sizeSpend.take(10)
println("Top 10 sizes by total spend:")
topSizes.foreach(println)
```

The Code:

```
Top 10 sizes by total spend:
(105167.0,M)
(61667.0,L)
(40468.0,S)
(25779.0,XL)
```

The Result:

- Regarding **colors**, the analysis of 25 unique shades pinpoints **Olive(177)**, **Yellow(174)**, and **Silver(173)** as the clear favorites. This strong customer preference provides a clear directive for visual strategy, with key recommendations including making these colors the focal point of in-store merchandising and utilizing them as the primary palette in digital marketing campaigns to maximize appeal.

```
val colorCounts = base
  .filter(r => nz(r.color).nonEmpty)
  .map(r => (nz(r.color), 1))
  .reduceByKey(_ + _)
  .map{ case (c, n) => (n, c) }
  .sortBy(_._1, ascending = false)

val topColors = colorCounts.take(10)
val nDistinctColors = base.map(r => nz(r.color)).distinct().count()
println(s"Distinct colors: $nDistinctColors")
println("Top 10 colors by frequency:")
topColors.foreach(println)
```

The Code:

```
Distinct colors: 25
Top 10 colors by frequency:
(177,Olive)
(174,Yellow)
(173,Silver)
(172,Teal)
(169,Green)
(167,Black)
(166,Violet)
(166,Cyan)
(159,Gray)
(158,Maroon)
```

The Result:

2) Seasonality

Methodologically, this result was derived using a sequence of Spark RDD transformations (**filter**, **map**, **combineByKey**, **mapValues**) to calculate the average spend per season, followed by a **takeOrdered** action to trigger the computation and retrieve the final ranked list.

- The average spending is slightly higher during the **Fall** and **Winter** seasons compared to Spring and Summer. This trend implies that customers tend to spend more during colder months, likely due to seasonal clothing or promotional cycles. To leverage this trend, strategic planning should focus on timing major campaigns and new collections for the Fall and Winter periods.

```
val avgSpendBySeason = base
  .filter(r => nz(r.season).nonEmpty)
  .map(r => (nz(r.season), (r.amountUSD, 1)))
  .combineByKey(
    (v: (Double, Int)) => (v._1, v._2),
    (acc: (Double, Int), v: (Double, Int)) => (acc._1 + v._1, acc._2 + v._2),
    (a: (Double, Int), b: (Double, Int)) => (a._1 + b._1, a._2 + b._2)
  )
  .mapValues{ case (sum, cnt) => sum / cnt }

val topSeasonsByAvg = avgSpendBySeason
  .takeOrdered(4)(Ordering.by[(String, Double), Double](_._2).reverse)
println("Seasons by average spend (desc):")
topSeasonsByAvg.foreach(println)
```

The Code:

```
Seasons by average spend (desc):
(Fall,61.55692307692308)
(Winter,60.35736354273944)
(Spring,58.73773773773774)
(Summer,58.405235602094244)
```

The Result:

3) Shipping Preferences & Distribution

Methodologically, a **countByValue** operation determined shipping distribution, while a series of transformations (**filter**, **map**, **reduceByKey**, **sortBy**) calculated spending by method.

```
scala> val shipDist = base.map(r => nz(r.shipping)).countByValue()
println("Shipping type distribution:")
shipDist.toSeq.sortBy(_._2).foreach(println)

val spendByShipping = base
  .filter(r => nz(r.shipping).nonEmpty)
  .map(r => (nz(r.shipping), r.amountUSD))
  .reduceByKey(_ + _)
  .map{ case (k, sum) => (sum, k) }
  .sortBy(_._1, ascending = false)

println("Spend by shipping type (desc):")
spendByShipping.take(10).foreach(println)
Shipping type distribution:
(Free Shipping,675)
(Standard,654)
(Store Pickup,650)
(Next Day Air,648)
(Express,646)
(2-Day Shipping,627)
Spend by shipping type (desc):
(40777.0,Free Shipping)
(39867.0,Express)
(38931.0,Store Pickup)
(38233.0,Standard)
(38080.0,2-Day Shipping)
(37993.0,Next Day Air)
val shipDist: scala.collection.Map[String,Long] = HashMap(Store Pickup -> 650, Standard -> 654, Free Shipping -> 675, Express -> 646, Next Day Air -> 648, 2-Day Shipping -> 627)
val spendByShipping: org.apache.spark.rdd.RDD[(Double, String)] = MapPartitionsRDD[27] at sortBy at <console>:10
```

Free Shipping dominates with 675 orders and the highest revenue at \$40,777, followed by Standard (654 orders, \$38,233) and Store Pickup (650 orders, \$38,931). Customers clearly prefer free shipping but will pay for expedited services when needed. Strategic recommendations include prominently featuring free shipping thresholds to increase order values and optimizing the Store Pickup experience to capitalize on this high-performing channel.

4) Product Categories & Item Preferences

Methodologically, item descriptions were tokenized via **flatMap**, normalized, and cleaned. Then map and **reduceByKey** generated word counts, followed by **sortBy** and take to extract the top 15 tokens.

```
scala> val tokens = base.flatMap { r =>
  nz(r.item).toLowerCase.replaceAll("[^a-z]", " ")
    .split("\\s+").filter(_.nonEmpty)
}

val wordCounts = tokens
  .map(w => (w, 1))
  .reduceByKey(_ + _)
  .map{ case (w, c) => (c, w) }
  .sortBy(_._1, ascending = false)

val topWords = wordCounts.take(15)
println("Top 15 item tokens:")
topWords.foreach(println)
Top 15 item tokens:
(316,shirt)
(171,pants)
(171,jewelry)
(171,blouse)
(160,dress)
(164,sweater)
(163,jacket)
(161,belt)
(161,coat)
(161,sunglasses)
(160,sandals)
(159,socks)
(158,skirt)
(157,shorts)
(157,scarf)
val tokens: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[28] at flatMap at <console>:1
val wordCounts: org.apache.spark.rdd.RDD[(Int, String)] = MapPartitionsRDD[34] at sortBy at <console>:9
val topWords: Array[(Int, String)] = Array((316,shirt), (171,pants), (171,jewelry), (171,blouse), (160,dress), (164,sweater), (163,jacket), (161,belt), (161,coat), (161,sunglasses), (160,sandals), (159,socks), (158,skirt), (157,shorts), (157,scarf))
```

Shirts lead decisively with 316 mentions, followed by pants (171), jewelry (171), and blouses (171). Accessories and outerwear maintain consistent presence (160-

166 mentions). The shirt category's dominance indicates it serves as a primary traffic driver. Key recommendations include expanding the shirt collection to maintain market leadership and creating outfit bundles combining high-frequency items to increase basket sizes.

5) Subscription Status Impact

Methodologically, this analysis used Spark RDD transformations (**map**, **aggregateByKey**, **mapValues**, **sortBy**) to calculate total spending, transaction **counts**, and **average** order values by subscription status, followed by a **collect** action

```
scala> val spendBySubscription = base
      |   .map(r => (nz(r.subscription), r.amountUSD))
      |   .aggregateByKey((0.0, 0))((acc, value) => (acc._1 + value, acc._2 + 1),
      |   (acc1, acc2) => (acc1._1 + acc2._1, acc1._2 + acc2._2))
      |   .mapValues { case (sum, count) => (sum, count, sum / count) }
      |   .sortBy(_._2._3)
      |
      |   println("Subscription Status Impact:")
      |   spendBySubscription.collect().foreach { case (status, (total, count, avg)) =>
      |     println(f"Status: $status, Count: $count, Total: $$${total}%.2f, Avg: $$${avg}%.2f")
      |   }
      | }
Subscription Status Impact:
Status: No, Count: 2847, Total: $170436.00, Avg: $59.87
Status: Yes, Count: 1053, Total: $62645.00, Avg: $59.49
val spendBySubscription: org.apache.spark.rdd.RDD[(String, (Double, Int, Double))] = MapPartitionsRDD[43] at sortBy at
<console>:8
```

The data shows non-subscribers dominate with 2,847 transactions generating \$170,436, while subscribers contribute only 1,053 transactions and \$62,645. However, average order values are nearly identical at \$59.87 versus \$59.49, revealing that subscription status does not drive higher individual spending. Only 27% of customers are subscribed, and they generate proportional revenue without premium spending behavior.

This indicates the subscription program maintains engagement but fails to incentivize larger purchases. Key recommendations include implementing tiered subscription models with exclusive perks to drive higher spending, increasing subscription adoption from 27% to 50%+ through targeted campaigns.

5. SQL Operations

Query 1: Top Products for Each Age Group

- To identify which products are the most popular among different age segments.

```
scala> :pa
// Entering paste mode (ctrl-D to finish)

println("==== Query 1: Top Products for Each Age Group ====")

spark.sql("""
SELECT
  CASE
    WHEN Age BETWEEN 18 AND 25 THEN '18-25'
    WHEN Age BETWEEN 26 AND 35 THEN '26-35'
    WHEN Age BETWEEN 36 AND 50 THEN '36-50'
    ELSE '50+'
  END AS Age_Group,
  'Item Purchased' AS Product,
  COUNT(*) AS total_purchases
FROM shopping
GROUP BY Age_Group, 'Item Purchased'
ORDER BY Age_Group, total_purchases DESC
""").coalesce(1).show(40, false)

// Exiting paste mode, now interpreting.

==== Query 1: Top Products for Each Age Group ====
+-----+-----+-----+
|Age_Group|Product      |total_purchases|
+-----+-----+-----+
|18-25    |Sweater      |31              |
|18-25    |Coat         |29              |
|18-25    |Dress        |29              |
|18-25    |Sneakers     |28              |
|18-25    |Skirt        |27              |
|18-25    |Shorts       |27              |
|18-25    |Pants        |26              |
|18-25    |Belt         |26              |
|18-25    |Shirt        |26              |
|18-25    |T-shirt      |25              |
|18-25    |Gloves       |24              |
|18-25    |Scarf        |23              |
|18-25    |Hat          |23              |
|18-25    |Blouse       |23              |
|18-25    |Socks        |23              |
|18-25    |Jacket       |22              |
|18-25    |Handbag      |21              |
|18-25    |Sunglasses   |20              |
|18-25    |Sandals      |19              |
|18-25    |Jewelry      |19              |
|18-25    |Boots        |18              |
|18-25    |Hoodie       |17              |
|18-25    |Backpack     |16              |
|18-25    |Jeans        |15              |
|18-25    |Shoes        |14              |
|26-35    |Shirt        |40              |
|26-35    |Backpack     |39              |
|26-35    |Jewelry      |38              |
|26-35    |Sunglasses   |34              |
|26-35    |Socks        |34              |
|26-35    |T-shirt      |33              |
|26-35    |Blouse       |33              |
|26-35    |Coat         |32              |
|26-35    |Shorts       |31              |
|26-35    |Gloves       |31              |
|26-35    |Hoodie       |31              |
|26-35    |Belt         |31              |
|26-35    |Skirt        |29              |
|26-35    |Shoes        |29              |
|26-35    |Jeans        |29              |
+-----+-----+-----+
only showing top 40 rows
```

- The 18–25 age group demonstrated the highest purchasing activity, primarily focused on fashion-oriented products such as sweaters, coats, dresses, sneakers, and skirts.
- The 26–35 segment showed a preference for versatile and stylish essentials such as backpacks, jewelry, and sunglasses, indicating a balance between practicality and personal style.
- This suggests that younger users (18–25) are trend-driven, while slightly older groups (26–35) lean toward functional yet fashionable purchases.

Query 2: Loyalty Score Analysis (Subscription × Previous Purchases)

- To analyze how subscription status affects customer loyalty based on previous purchases and spending.

```
scala> :pa
// Entering paste mode (ctrl-D to finish)

println("==== Query 2: Loyalty Score Analysis (Subscription # Previous Purchases) =====")
spark.sql("""
SELECT
  'Subscription Status',
  ROUND(AVG('Previous Purchases' * 'Purchase Amount (USD)'), 2) AS loyalty_score
FROM shopping
GROUP BY 'Subscription Status'
ORDER BY loyalty_score DESC
""").show()

// Exiting paste mode, now interpreting.

==== Query 2: Loyalty Score Analysis (Subscription # Previous Purchases) =====
+-----+-----+
|Subscription Status|loyalty_score|
+-----+-----+
|Yes|1551.81|
|No|1505.32|
+-----+-----+
```

- Customers with active subscriptions achieved a higher loyalty score (1551.81) compared to non-subscribers (1505.32). This suggests that subscription members purchase more frequently and spend more overall.
- Insight: Subscription programs positively impact repeat purchases and customer retention, making them a strong loyalty driver.

Query 3: Relationship Between Review Rating, Season, and Discount

- To explore how discounts and seasonal changes affect customer ratings and spending behavior.

```
scala> :pa
// Entering paste mode (ctrl-D to finish)

println("==== Query 3: Relationship Between Review Rating, Season, and Discount =====")
spark.sql("""
SELECT
  Season,
  'Discount Applied',
  ROUND(AVG('Review Rating'), 2) AS avg_rating,
  ROUND(AVG('Purchase Amount (USD)'), 2) AS avg_spending
FROM shopping
GROUP BY Season, 'Discount Applied'
ORDER BY avg_rating DESC
""").show()

// Exiting paste mode, now interpreting.

==== Query 3: Relationship Between Review Rating, Season, and Discount =====
+-----+-----+-----+-----+
|Season|Discount Applied|avg_rating|avg_spending|
+-----+-----+-----+-----+
|Spring|Yes|3.8|58.18|
|Spring|No|3.79|59.18|
|Fall|No|3.76|61.53|
|Winter|Yes|3.75|60.69|
|Winter|No|3.75|60.11|
|Summer|No|3.73|59.64|
|Summer|Yes|3.72|56.86|
|Fall|Yes|3.69|61.59|
+-----+-----+-----+-----+
```

- During the Spring season, products with discounts received the highest average rating (3.8) and consistent spending around 58–60 USD. In contrast, Fall and

Winter seasons without discounts showed similar satisfaction levels but slightly higher spending.

- Offering discounts during key seasons (like Spring and Summer) increases customer satisfaction and boosts purchase volume.

Query 4: High-Value Customers Profile

- To determine which gender group contributes more to high-value purchases.

```
scala> :pa
// Entering paste mode (ctrl-D to finish)

println("==== Query 4: High-Value Customers Profile ====")
spark.sql("""
SELECT
  Gender,
  ROUND(AVG('Purchase Amount (USD)'),2) AS avg_spending,
  ROUND(AVG(Age),0) AS avg_age
FROM shopping
GROUP BY Gender
HAVING AVG('Purchase Amount (USD)') >
  (SELECT AVG('Purchase Amount (USD)') FROM shopping)
ORDER BY avg_spending DESC
""").show()

// Exiting paste mode, now interpreting.

==== Query 4: High-Value Customers Profile ====
+-----+-----+-----+
|Gender|avg_spending|avg_age|
+-----+-----+-----+
|Female|      60.25|    44.0|
+-----+-----+-----+
```

- Female customers had an average spending of 60.25 USD with an average age of 44, surpassing the dataset's global spending average.
- Women represent the platform's high-value demographic, and marketing efforts should target this group with premium offers and loyalty incentives.

Query 5: Payment Method × Shipping Type

- To analyze the relationship between payment methods and preferred shipping options.

```
// Entering paste mode (ctrl-D to finish)

println("==== Query 5: Payment Method × Shipping Type =====")
spark.sql("""
SELECT
  CASE
    WHEN 'Payment Method_Credit Card' = 1 THEN 'Credit Card'
    WHEN 'Payment Method_Debit Card' = 1 THEN 'Debit Card'
    WHEN 'Payment Method_Cash' = 1 THEN 'Cash'
    WHEN 'Payment Method_PayPal' = 1 THEN 'PayPal'
    WHEN 'Payment Method_Venmo' = 1 THEN 'Venmo'
    ELSE 'Other'
  END AS Payment_Method,
  'Shipping Type',
  COUNT(*) AS total_orders,
  ROUND(AVG('Purchase Amount (USD)'),2) AS avg_spending
FROM shipping
GROUP BY Payment_Method, 'Shipping Type'
ORDER BY total_orders DESC
""").show()

// Exiting paste mode, now interpreting.

==== Query 5: Payment Method × Shipping Type =====
+-----+-----+-----+-----+
|Payment_Method|Shipping Type|total_orders|avg_spending|
+-----+-----+-----+-----+
|PayPal|Express|129|59.65|
|PayPal|Free Shipping|123|59.65|
|Credit Card|2-Day Shipping|123|60.25|
|Cash|Free Shipping|121|60.17|
|Cash|Store Pickup|119|60.29|
|Cash|Standard|118|57.65|
|Debit Card|Standard|118|60.64|
|Debit Card|Free Shipping|117|61.39|
|Other|Next Day Air|117|56.84|
|Credit Card|Standard|115|56.88|
|Credit Card|Express|114|60.73|
|Other|Store Pickup|112|57.79|
|PayPal|2-Day Shipping|109|58.82|
|Venmo|Next Day Air|108|58.44|
|Credit Card|Next Day Air|108|61.43|
|Cash|Express|108|62.23|
|Venmo|Store Pickup|107|56.75|
|PayPal|Standard|107|58.54|
|Credit Card|Free Shipping|107|60.06|
|Venmo|Free Shipping|106|59.54|
+-----+-----+-----+-----+
only showing top 20 rows
```

- The most common combinations were PayPal + Express Shipping (129 orders) and Credit Card + 2-Day Shipping (123 orders), both showing average spending between 59–62 USD.
- Customers who use online payment methods (PayPal, Credit Card) tend to select faster shipping options, indicating a preference for convenience and reliability.

6. Machine Learning Operations

Data Preprocessing Pipeline

Methodologically, a `SparkSession` was initialized to load the shopping behavior dataset from a CSV file with automatic schema inference and header recognition. The preprocessing pipeline segregated features into categorical variables (Gender, Category, Payment Method, Shipping Type, Season, Size) and numerical variables (Age, Previous Purchases, Review Rating), with missing values systematically imputed using "Unknown" for categorical fields and 0 for numerical fields.

```
val spark = SparkSession.builder().appName("ShoppingML").getOrCreate()

val path = "/Users/mac/Downloads/shopping_behavior_cleaned (1).csv"
var df = spark.read.option("header", "true").option("inferSchema", "true").csv(path)

val catCols = Seq("Gender", "Category", "Payment Method", "Shipping Type", "Season", "Size").filter(df.columns.contains)
val numCols = Seq("Age", "Previous Purchases", "Review Rating").filter(df.columns.contains)

df = df.na.fill("Unknown", catCols).na.fill(0, numCols)

val indexers = catCols.map(c => new StringIndexer().setInputCol(c).setOutputCol(c + "_idx").setHandleInvalid("keep"))
val encoders = catCols.map(c => new OneHotEncoder().setInputCol(c + "_idx").setOutputCol(c + "_ohe"))
val assemblerInputs = numCols ++ catCols.map(_ + "_ohe")
val assembler = new VectorAssembler().setInputCols(assemblerInputs.toArray).setOutputCol("features_raw")
val scaler = new StandardScaler().setInputCol("features_raw").setOutputCol("features").setWithMean(true).setWithStd(true)

25/11/06 17:33:21 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._
import org.apache.spark.ml.Pipeline
import org.apache.spark.ml.feature._
import org.apache.spark.ml.linalg.Vector
val spark: org.apache.spark.sql.SparkSession = org.apache.spark.sql.classic.SparkSession@357d787b
val path: String = /Users/mac/Downloads/shopping_behavior_cleaned (1).csv
var df: org.apache.spark.sql.DataFrame = [Age: int, Gender: string ... 87 more fields]
val catCols: Seq[String] = List(Gender, Shipping Type, Season, Size)
val numCols: Seq[String] = List(Age, Previous Purchases, Review Rating)
// mutated df
val indexers: Seq[org.apache.spark.m...
```

Feature transformation employed a three-stage pipeline:

StringIndexer converted categorical strings to numeric indices with invalid value handling, **OneHotEncoder** created binary indicator vectors (e.g., "Gender_ohe", "Category_ohe"), and **VectorAssembler** consolidated all encoded features into a unified "features_raw" vector. **StandardScaler** then normalized this vector (mean=true, std=true) to produce the final "features" column, ensuring all variables operated on comparable scales for downstream machine learning algorithms.

Purchase Amount Prediction using Random Forest Regression

Methodologically, a Random Forest Regressor (100 trees) was trained to predict **Purchase Amount (USD)** after removing null values from the target variable. The pipeline integrated all preprocessing stages (indexers, encoders, assembler, scaler) with the regression model, utilizing a 75-25 train-test split with seed 42 for reproducibility.

```
scala> import org.apache.spark.ml.regression.RandomForestRegressor
import org.apache.spark.ml.evaluation.RegressionEvaluator

val target = "Purchase Amount (USD)"
df = df.na.fill(0, Seq(target))

val rf = new RandomForestRegressor().setLabelCol(target).setFeaturesCol("features").setNumTrees(100)
val pipeline = new Pipeline().setStages((indexers ++ encoders ++ assembler ++ scaler ++ rf).toArray)

val Array(train, test) = df.randomSplit(Array(0.75, 0.25), 42)
val model = pipeline.fit(train)
val preds = model.transform(test)

val rmse = new RegressionEvaluator().setLabelCol(target).setPredictionCol("prediction").setMetricName("rmse").evaluate(preds)
val r2 = new RegressionEvaluator().setLabelCol(target).setPredictionCol("prediction").setMetricName("r2").evaluate(preds)

println(rmse, r2)
25/11/06 17:33:39 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
(23.646854973668686,-0.085388878590655851)
import org.apache.spark.ml.regression.RandomForestRegressor
import org.apache.spark.ml.evaluation.RegressionEvaluator
val target: String = Purchase Amount (USD)
// mutated df
val rf: org.apache.spark.ml.regression.RandomForestRegressor = rfg_07156b933bf7
val pipeline: org.apache.spark.ml.Pipeline = pipeline_2638dd89000b
val train: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [Age: int, Gender: string ... 87 more fields]
val test: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [Age: int, Gender: string ... 87 more fields]
val model: org.apache.spark.ml.PipelineModel = pipeline_2638dd89000b
val preds: ...
```

Model performance was evaluated using RegressionEvaluator across two metrics:

RMSE of 23.65 and **R² of -0.085**, indicating the model explains virtually none of the variance in purchase amounts and performs worse than a simple mean baseline. suggest that purchase amount may be driven by factors not captured in the available features, or that the relationship between features and spending is highly non-linear, requiring alternative modeling approaches such as gradient boosting or deep learning architecture.

Subscription Status Classification using Random Forest

Methodologically, a Random Forest Classifier (100 trees) was trained to predict **Subscription Status** using the same feature engineering pipeline employed in the regression model. The complete ML pipeline incorporated StringIndexer for label encoding, followed by the full feature transformation stack (indexers, encoders, assembler, scaler), culminating in the RandomForestClassifier. A 75-25 train-test split with seed 42 ensured reproducibility.

```
scala> import org.apache.spark.ml.classification.RandomForestClassifier
import org.apache.spark.ml.evaluation.MulticlassClassificationEvaluator

val labelCol = "Subscription Status"
df = df.na.fill("Unknown", Seq(labelCol))

val labelIndexer = new StringIndexer().setInputCol(labelCol).setOutputCol("label").setHandleInvalid("keep")
val rfClf = new RandomForestClassifier().setLabelCol("label").setFeaturesCol("features").setNumTrees(100)

val pipelineClf = new Pipeline().setStages((indexers ++ encoders ++ assembler ++ scaler ++ labelIndexer ++ rfClf).toArray)
val Array(trn, tst) = df.randomSplit(Array(0.75, 0.25), 42)
val modelClf = pipelineClf.fit(trn)
val predsClf = modelClf.transform(tst)

val acc = new MulticlassClassificationEvaluator().setLabelCol("label").setPredictionCol("prediction").setMetricName("accuracy").evaluate(predsClf)
println(acc)

predsClf.groupBy("label", "prediction").count().show(false)
0.7388108108108108
+-----+
|label|prediction|count|
+-----+
|1.0  |0.0       |249  |
|8.0  |0.0       |676  |
+-----+
```

```
import org.apache.spark.ml.classification.RandomForestClassifier
import org.apache.spark.ml.evaluation.MulticlassClassificationEvaluator
val labelCol: String = Subscription Status
// mutated df
val labelIndexer: org.apache.spark.ml.feature.StringIndexer = strIdx_e4a4f4fbc8b
val rfClf: org.apache.spark.ml.classification.RandomForestClassifier = rfc_a2350939e518
val pipelineClf: org.apache.spark.ml.Pipeline = pipeline_3abba1fb7ac85
val trn: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [Age: int, Gender: string ... 87 more fields]
val tst: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [Age: int, Gender: string .....
```

Model performance achieved **73.08% accuracy** on the test set, as measured by MulticlassClassificationEvaluator. The partial confusion matrix displayed shows:

- **676 True Negatives:** Non-subscribers correctly identified as inactive
- **249 False Negatives:** Subscribers incorrectly classified as inactive

The results indicate that the model effectively identifies most non-subscribers, correctly classifying 676 inactive customers. However, the 249 misclassified subscribers highlight a key limitation, as these false negatives represent potential revenue loss from customers who were incorrectly labeled as uninterested.

Customer Segmentation using K-Means Clustering

Methodologically, K-Means clustering (k=3, seed=42) was applied to the scaled feature space to identify distinct customer segments. The algorithm produced three clusters with notably different behavioral profiles based on aggregated statistics.

```
scala> import org.apache.spark.ml.clustering.KMeans

val pipelineFeat = new Pipeline().setStages((indexers ++ encoders ++ assembler ++ scaler).toArray)
val featModel = pipelineFeat.fit(df)
val dfFeat = featModel.transform(df)

val kmeans = new KMeans().setK(3).setSeed(42).setFeaturesCol("features").setPredictionCol("cluster")
val kModel = kmeans.fit(dfFeat)
val clustered = kModel.transform(dfFeat)

clustered.groupBy("cluster").agg(count("*").alias("size"), avg(target).alias("avg_purchase")).show(false)
25/11/06 17:36:19 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.blas.JNIBLAS
+-----+
|cluster|size|avg_purchase|
+-----+
|1|1663|60.2976484864943|
|2|1248|60.249298737948715|
|0|989|58.25581395348837|
+-----+

import org.apache.spark.ml.clustering.KMeans
val pipelineFeat: org.apache.spark.ml.Pipeline = pipeline_e0ae0163bf1e
val featModel: org.apache.spark.ml.PipelineModel = pipeline_e0ae0163bf1e
val dfFeat: org.apache.spark.sql.DataFrame = [Age: int, Gender: string ... 97 more fields]
val kmeans: org.apache.spark.ml.clustering.KMeans = kmeans_373e65dc45ee
val kModel: org.apache.spark.ml.clustering.KMeansModel = KMeansModel: uid=kmeans_373e65dc45ee, k=3, distanceMeasure=euclidean, numFeatures=19
val clustered: org.apache.spark.sql.DataFrame = [Age: int, Gender: string ... 98 more fields]
```

Cluster characteristics:

- **Cluster 1** (1663 customers): Average purchase of \$60.30
- **Cluster 2** (1248 customers): Average purchase of \$60.25
- **Cluster 0** (989 customers): Average purchase of \$58.26

The similar spending levels among clusters indicate that customers differ more in their demographic and behavioral traits such as gender, season, or purchase preferences rather than in how much they spend. This suggests that future segmentation could benefit from incorporating lifestyle or engagement variables to reveal deeper behavioral patterns.

Advanced Customer Profiling using Gaussian Mixture Model

Methodologically, a refined clustering pipeline was constructed using only numerical features (Age, Previous Purchases, Review Rating, Purchase Amount USD). A VectorAssembler created the feature vector, StandardScaler normalized the data, and K-Means (k=4, seed=42) produced four distinct customer segments with comprehensive statistical profiles.

```
import org.apache.spark.sql.functions._

val clusterCols = Seq("Age", "Previous Purchases", "Review Rating", "Purchase Amount (USD)").filter(df.columns.contains)

val dfNum = clusterCols.foldLeft(df[(id, c) => d.withColumn(c, col(c).cast("double"))])
val assembler = new VectorAssembler()
    .setInputCols(clusterCols.toArray)
    .setOutputCol("features_raw")

val dfVec = assembler.transform(dfNum)

val scaler = new StandardScaler()
    .setInputCol("features_raw")
    .setOutputCol("features")
    .setWithStd(true)
    .setWithMean(true)

val dfScaled = scaler.fit(dfVec).transform(dfVec)

val kmeans = new KMeans().setK(4).setSeed(42).setFeaturesCol("features").setPredictionCol("cluster")
val model = kmeans.fit(dfScaled)
val clustered = model.transform(dfScaled)

val summary = clustered.groupBy("cluster")
    .agg(
        count("id").as("count"),
        avg("Age").as("avg_age"),
        avg("Purchase Amount (USD)").as("avg_spend"),
        avg("Previous Purchases").as("avg_prev"),
        avg("Review Rating").as("avg_rating")
    )
    .orderBy("cluster")

summary.show(false)

+-----+
|cluster|count|avg_age|avg_spend|avg_prev|avg_rating|
+-----+
|0      |984   |39.52  |$57.22   |11.97   |4.32      |
|1      |1000  |48.42  |$37.38   |26.74   |3.20      |
|2      |966   |46.11  |$64.88   |39.15   |4.34      |
|3      |958   |42.12  |$80.78   |23.72   |3.15      |
+-----+

import org.apache.spark.ml.feature.{VectorAssembler, StandardScaler}
import org.apache.spark.ml.clustering.KMeans
import org.apache.spark.sql.functions._

val clusterCols: Seq[String] = List(Age, Previous Purchases, Review Rating, Purchase Amount (USD))
val dfNum = org.apache.spark.sql.Dataset.fromSeq(
    Seq(
      (1, 39.52, 57.22, 11.97, 4.32),
      (2, 48.42, 37.38, 26.74, 3.20),
      (3, 46.11, 64.88, 39.15, 4.34),
      (4, 42.12, 80.78, 23.72, 3.15)
    )
).toDF("id", "Age", "Purchase Amount (USD)", "Previous Purchases", "Review Rating")
val assembler = new VectorAssembler()
    .setInputCols(clusterCols.toArray)
    .setOutputCol("features")
val scaler = new StandardScaler()
    .setInputCol("features")
    .setOutputCol("features_scaled")
    .setWithStd(true)
    .setWithMean(true)
val dfScaled = scaler.fit(assembler.transform(dfNum)).transform(dfScaled)
val kmeans = new KMeans()
    .setK(4)
    .setSeed(42)
    .setFeaturesCol("features_scaled")
    .setPredictionCol("cluster")
val model = kmeans.fit(dfScaled)
val clustered = model.transform(dfScaled)
```

Cluster profiles:

- **Cluster 0** (984 customers): 39.52 avg age, \$57.22 spend, 11.97 previous purchases, 4.32 rating
- **Cluster 1** (1,000 customers): 48.42 avg age, \$37.38 spend, 26.74 previous purchases, 3.20 rating
- **Cluster 2** (966 customers): 46.11 avg age, \$64.88 spend, 39.15 previous purchases, 4.34 rating
- **Cluster 3** (958 customers): 42.12 avg age, \$80.78 spend, 23.72 previous purchases, 3.15 rating

This analysis reveals four clear customer archetypes with distinct behavioral patterns. **Cluster 2** reflects highly engaged, loyal customers with strong spending and satisfaction levels, while **Cluster 3** represents high-value but less frequent buyers. **Cluster 1** includes frequent yet low-value purchasers, and **Cluster 0** corresponds to newer or less engaged users. Overall, the segmentation highlights distinct engagement profiles rather than a continuous spending spectrum.

Feature Importance Analysis using Random Forest

Methodologically, feature importance scores were extracted from both the trained Random Forest Regression and Classification models, sorted in descending order, with the top 15 features displayed for each model.

```
scala> val featCols = assemblerInputs.toArray

val rfModelReg = model.stages.last.asInstanceOf[org.apache.spark.ml.regression.RandomForestRegressionModel]
val rfModelCls = model.stages.last.asInstanceOf[org.apache.spark.ml.classification.RandomForestClassificationModel]

val importancesReg = rfModelReg.featureImportances.toArray.zip(featCols).sortBy(_._1)
val importancesCls = rfModelCls.featureImportances.toArray.zip(featCols).sortBy(_._1)

println("Top Regression Features:")
importancesReg.take(15).foreach(println)

println("Top Classification Features:")
importancesCls.take(15).foreach(println)

Top Regression Features:
(0.28687587956464096,Previous Purchases)
(0.2034782953596182,Review Rating)
(0.1992714567154396,Age)
(0.026684573318203675,Gender_ohe)
(0.021434359138611626,Season_ohe)
(0.019913286526391777,Shipping Type_ohe)
(0.017567923498127448,Size_ohe)
Top Classification Features:
(0.4119856157128387,Gender_ohe)
(0.39723573498997483,Shipping Type_ohe)
(0.0432865372798437,Previous Purchases)
(0.04243475686245757,Review Rating)
(0.03255836492964485,Age)
(0.007258382244647645,Size_ohe)
(0.002826321757855398,Season_ohe)
val featCols: Array[String] = Array(Age, Previous Purchases, Review Rating, Gender_ohe, Shipping Type_ohe, Season_ohe, Size_ohe)
val rfModelReg: org.apache.spark.ml.regression.RandomForestRegressionModel = RandomForestRegressionModel: uid=rfc_071558933b77, numTrees=100, numFeatures=19
val rfModelCls: org.apache.spark.ml.classification.RandomForestClassificationModel = RandomForestClassificationModel: uid=rfc_a2358939e518, numTrees=100, numClasses=3, numFeatures=19
val importancesReg: Array[(Double, String)] = Array((0.28687587956464096,Previous Purchases), (0.2034782953596182,Review Rating), (0.1992714567154396,Age), (0.026684573318203675,Gender_ohe), (0.021434359138611626,Season_ohe)...
```

Regression model top predictors:

1. Previous Purchases (0.206) - strongest predictor of purchase amount
2. Review Rating (0.203) - customer satisfaction correlates with spending
3. Age (0.199) - demographic factor influencing purchase value
4. Gender_ohe (0.026) and Season_ohe (0.021) - minor categorical effects

Classification model top predictors:

1. Gender_ohe (0.411) - gender is the strongest subscription predictor
2. Shipping Type_ohe (0.397) - delivery preferences indicate subscription likelihood
3. Previous Purchases (0.043) - purchase history dominates subscription prediction
4. Review Rating (0.042) - satisfaction influences subscription decisions

Overall, purchase-related behavior is primarily driven by historical engagement and satisfaction, while subscription likelihood depends heavily on demographic and logistical preferences such as gender and shipping type. This indicates that personalization strategies should emphasize user experience and delivery preferences to improve both purchase value and subscription retention.